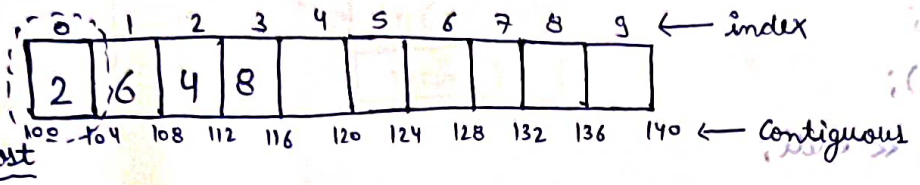
Lecture ⑨

Arrays

- Collection of same type of items
- memory stored at contiguous location
- elements can be accessed by index
- Faayda → Multiple values ko single variable me store kar sakte Hai

Declaration :-

```
int dost[10];
```



Array ka name Bhi Hai aur first location ka bhi darsha raha hai

dost[0] → 1st location ⇒ 2

i.e. cout << dost[0]; → 2

dost[3] = 100 + 3×4 = 112 (location) ⇒ 8 (value at index 3)

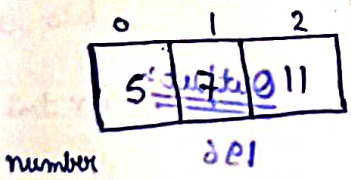
(int bytes)

Array (n size) → Index ⇒ 0 to (n-1)

⇒ For deleting a dynamically allocated array ⇒ delete [] arr;

Initialisation :-

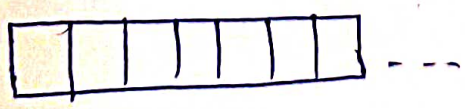
```
int number[3] = {5, 7, 11}
```



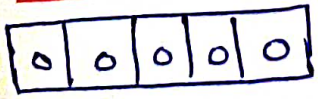

```
{ Dynamic array
int *arr = new int[n]; }
```

Ex:- ① `int array[100000];`

garbage values

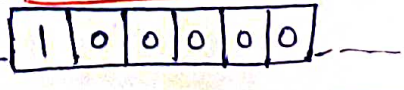


② `int array[10000] = {0};`



③ `int array[100000] = {1};`

} ✗ It will be



Qu- Entire array ko kisi bhi value se initialize kaise karu?

↳ ① By using for loop
↳ ② By using fill_n (arr, n, val)

Arrays with functions

```
void printArray (int arr[] , int size)
{
    for (int i=0 ; i < size; i++) {
        cout << arr[i] << " ";
    }
}
```

sizeof Array = sizeof(arr) / sizeof (int);

↳ But it has a problem

Ex:- int third [15] = {2, 7};

↳ so, sizeof() gives the length is not 2

Qu- Finding maximum and minimum element in an array

```
# include <iostream>
using namespace std;

void initializingArray (int arr[] , int size) {
    for (int i=0; i< size ; i++) {
        cout << " Enter index " << i << endl;
        cin >> arr[i];
    }
}

void printArray (int arr[], int size) {
    cout << " The given array is - " << endl;
    for (int i=0 ; i < size ; i ++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    cout << " Printing complete " << endl;
}
```

```
int printMaxim(int arr[], int size) {
    int K = arr[0];
    for (int i = 0; i < size; i++) {
        if (K < arr[i]) {
            K = arr[i];
        }
    }
    return K;
}

int printminim(int arr[], int size) {
    int l = arr[0];
    for (int i=0; i<size; i++){
        if (l > arr[i]) {
            l = arr[i];
        }
    }
    return l;
}

int main() {
    int n;
    cout << "Enter the size of Array" << endl;
    cin >> n;
    int array[n];
    initializingArray(array, n);
    printArray(array, n);
    int maxim = printmaxim(array, n);
    int minim = printminim(array, n);
    cout << "The maximum element is " << maxim << endl;
    cout << "The minimum element is" << minim << endl;
    return 0;
}
```

We can also use predefined function max:

$\Rightarrow K = max(K, arr[i]);$

predefined function for minimum:

$\Rightarrow l = min(l, arr[i]);$

$\Longrightarrow$ Bad practice (To take variable size in an array)

⇒

```
# include <iostream>
using namespace std;
void update (int arr[], int n) {
    cout << "Inside the function" << endl;
    arr[0] = 120;
    for (int i=0; i<n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    cout << "Going back to main function" << endl;
}
int main() {
    int arr[3] = {1,2,3};
    update (arr, 3);
    cout << "Printing in main function" << endl;
    for (int i=0; i<3; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

Output:-

```
Inside the function
120 2 3
Going back to main function
Printing in main function
120 2 3
```

Qu- Why arr[0] changed in main function ?

update (int arr[], int n)

update (arr, 3)

↳ we know that this arr signifies array & first location

So, maine pehle wale ka address update ko de diya hai, To jab ham koi bhi processing array me karenge To wo actual array me hi hogi kyoki address same hoga.

Qu- I/P → size & elements , O/P → sum of elements of array

Linear Search

```
# include <iostream>
using namespace std;
bool search(int arr[], int size, int key) {
    for(int i=0; i < size; i++) {
        if (arr[i] == key) {
            return 1;
        }
    }
    return 0;
}
int main() {
    int arr[10] = { 5, 7, -2, 10, 22, -2, 0, 5, 22, 1 };
    cout << " Enter the element to search for " << endl;
    int key;
    cin >> key;
    bool found = search(arr, 10, key);
    if (found) {
        cout << " key is present " << endl;
    }
    else {
        cout << " Key is absent " << endl;
    }
}
```

Qu- Reverse an Array

```
#include <iostream>
using namespace std;
void reverse(int arr[], int n) {
    int start = 0;
    int end = n-1;
    while (start <= end) {
        swap(arr[start], arr[end]);
        start++;
        end--;
    }
}
void printArray(int arr[], int n) {
    for (int i=0; i<n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
int main() {
    int arr[6] = {1, 4, 0, 5, -2, 15};
    int brr[5] = {2, 6, 3, 9, 4};
    reverse(arr, 6);
    reverse(brr, 5);
    printArray(arr, 6);
    printArray(brr, 5);
    return 0;
}
```

Qu - Swap alternate , ex:- {1, 2, 3, 4, 5, 6}
O/P ⇒ {2, 1, 4, 3, 6, 5}

Qu- Find unique element in array

Qu- Find duplicate element in an array

Qu- Intersection of two arrays

Qu- Pair sum

Qu- Triplet sum

Qu- sort 0's and 1's

Lecture - (10)

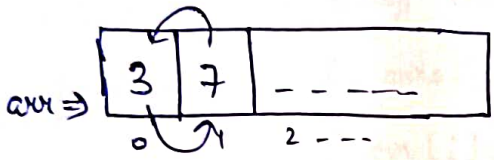
Qu- Swap alternate , I/P ⇒ {1,2,3,4,5,6} & O/P ⇒ {2,1,4,3,6,5}
I/P ⇒ {1,2,7,8,5} & O/P ⇒ {2,1,8,7,5}

```
#include <iostream>
using namespace std;

void printArray(int arr[], int n) {
    for (int i=0; i<n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void swapAlternate(int arr[], int size) {
    for (int i=0; i<size; i+=2) {
        if (i+1 < size) {
            swap(arr[i], arr[i+1]);
        }
    }
}
```

```
int main() {
    int even[8] = {5, 2, 9, 4, 7, 6, 1, 0};
    int odd[5] = {11, 33, 9, 76, 43};

    swapAlternate(even, 8);
    printArray(even, 8);
    cout << endl;
    swapAlternate(odd, 5);
    printArray(odd, 5);

    return 0;
}
```

Internal working of swap function

Ex-




```
temp = arr[1]
arr[1] = arr[0]
arr[0] = temp
```

temp → 7

Qu- Find unique element in array (Odd no. of elements i.e. (2m+1) & each element is in pair except one unique element)

⇒ $a \wedge a = 0$

⇒ $0 \wedge a = a$

Basically Hame vo number Kaatne Hai jo repeat Ho rahe hai

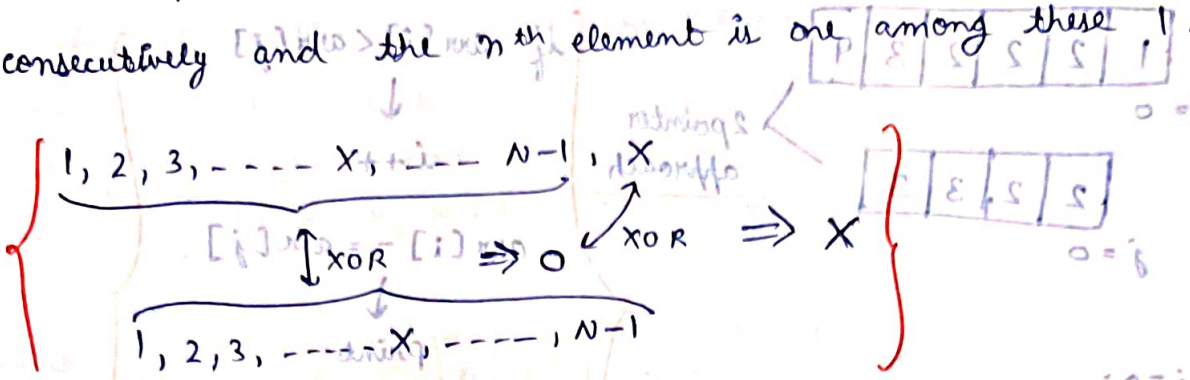
```
int findUnique(int arr[], int size) {
    int ans = 0;

    for(int i=0; i<size; i++) {

        ans = ans ^ arr[i];

    }

    return ans;
}
```

My soln

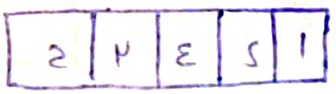
```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout<<"Enter the size of array"<<endl;
    cin>>n;
    int arr[n];
    cout<<"Enter the elements of array"<<endl;
    for(int i=0; i<n; i++) {
        cin>>arr[i];
    }
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            if(i!=j && arr[i]==arr[j]) {
                arr[i]=0;
                arr[j]=0;
            }
        }
    }
    for(int i=0; i<n; i++) {
        if(arr[i]!=0) {
            cout<<"The unique element is "<<arr[i];
        }
    }
    return 0;
}
```

Leetcode 1207 (Unique number of occurrences), Leetcode 442 (Find all duplicates in array)

Qu- Find duplicates in array (Total n elements , elements from 1 to n-1 consecutively and the n^{th} element is one among these 1 to n-1)




```
int findDuplicate(vector<int>& arr)
{
    int ans=0;
    // XOR ing all array elements
    for(int i=0; i<arr.size(); i++){
        ans = ans^arr[i];
    }

    // XOR [1, n-1]
    for(int i=1; i<arr.size(); i++) {
        ans = ans^i;
    }
    return ans;
}
```

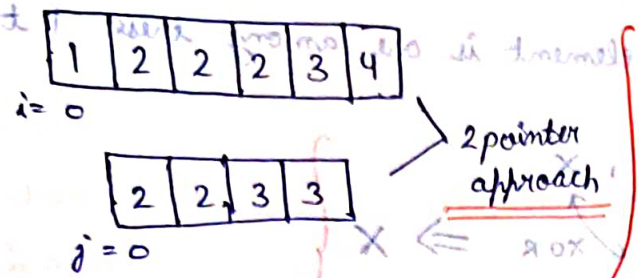


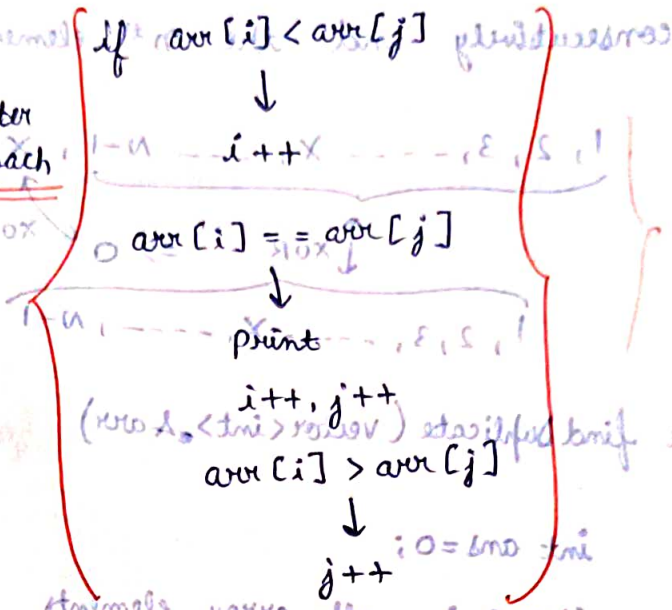
Qu- Intersection of two arrays (if not present return -1)
Arrays sorted in non-dec order

```
for(int i=0; i<n; i++) {
    int element = arr1[i];
    for(int j=0; j<m; j++) {
        if(element == arr2[j]) {
            cout<<element<<" ";
            arr2[j] = INT_MIN;
            break;
        }
    }
```

Optimised solⁿ

↳ By using the statement that arrays are sorted






```
int i=0, j=0;
Vector<int> ans;
while (i<n && j<m) {

  if (arr1[i] == arr2[j])
  {
      ans.push_back(arr1[i]);
      i++;
      j++;
  }
  else if (arr1[i] < arr2[j]) {
      i++;
  }
  else {
      j++;
  }
```

Qu- Pair Sum, Array → | 1 | 2 | 3 | 4 | 5 |

S = 5

Aise pair dhoondke do jinka Sum S ke barabar ho

Return the pairs in non-dec. order

```
for (int i=0; i<arr.size(); i++) {
    for (int j=i+1; j<arr.size(); j++) {
        if (arr[i] + arr[j] == S) {
            Vector<int> temp;
            temp.push_back(min(arr[i], arr[j]));
            temp.push_back(max(arr[i], arr[j]));
            ans.push_back(temp);
        }
    }
}
```

```
Sort (ans.begin(), ans.end());
return ans;
```

Qu- Triplets with given sum (Same as previous question)

Now in this question we will run 3 loops

① → i=0; i<n; i++

② → j=i+1; j<n; j++

③ → k=j+1; k<n; k++

Qu- Sort 0,1

I/P ⇒ | 0 | 1 | 1 | 0 | 0 | 1 |

O/P ⇒ | 0 | 0 | 0 | 1 | 1 | 1 |

Sol^n

↳ Counting
0 → 3
1 → 3
Traverse 0/1

↳ Sort

| 0 | 0 | 0 | 1 | 1 | 1 |

↳ Two pointer approach

↓

single traverse

0 → left
1 → right

| 0 | 1 | 0 | 1 | 1 | 0 |

i j

① arr[i] == 0 && i<j
↳ i++

② arr[j] == 1 && i<j
↳ j--

③ arr[i] == 1 && arr[j] == 0 && i<j
↳ ↙
swap(arr[i], arr[j])
i++;
j--

Dry Run

| 0 | 1 | 0 | 1 | 1 | 0 |

⇒ | 0 | 1 | 0 | 1 | 1 | 0 |

⇒ | 0 | 0 | 0 | 1 | 1 | 1 |

| 0 | 0 | 0 | 1 | 1 | 1 |

loop rok dena
cond^n ⇒ (i < j)

H.W.

Qu- Sort 0,1,2

I/P ⇒ | 0 | 1 | 2 | 2 | 1 | 0 | 2 |

O/P ⇒ | 0 | 0 | 1 | 1 | 2 | 2 | 2 |

Lecture (11)

Time & Space Complexity

Time complexity → It is the amount of time taken by an algorithm to run as a function of length of the input

Why? → For making better Programs
→ Comparison of Algorithm

→ **Big O Notation** → Upper Bound

→ **Theta Θ** → For avg. case complexity

→ **Omega Ω** → Lower Bound

Constant time → O(1) Ex:- for (int i=0; i<10; i++)

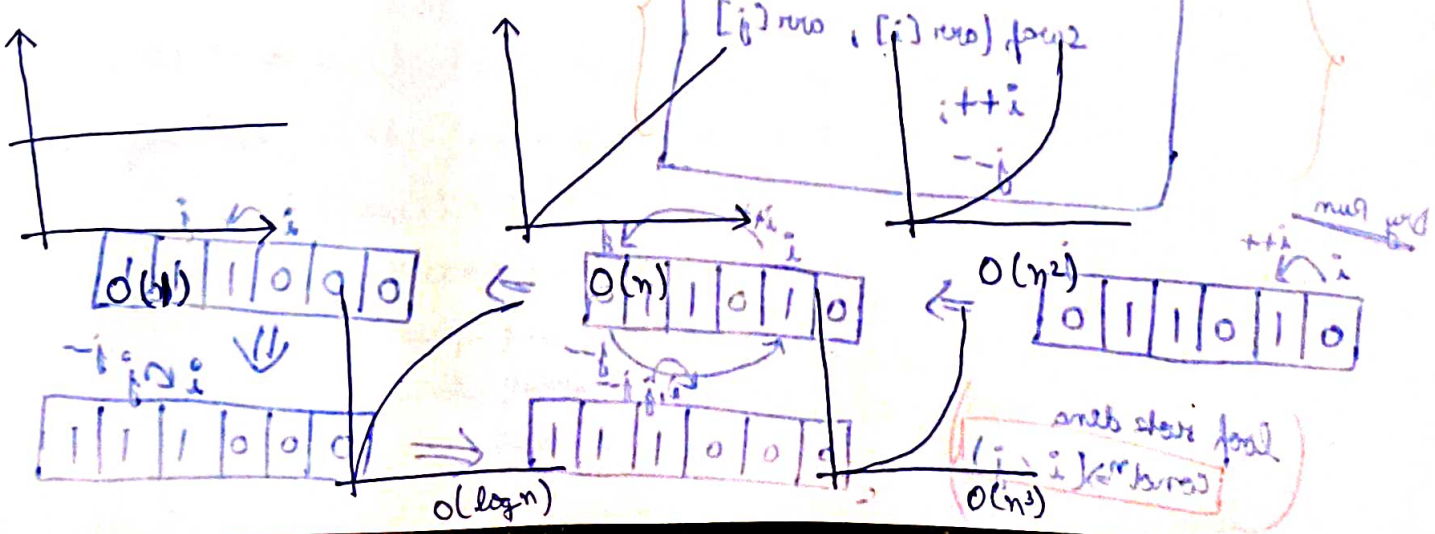
Linear time → O(n) Ex:- for (i=0; i<n; i++)

Logarithmic time → O(log n) Ex:- Binary search | 0 | 1 | 1 | 0 | 1 | 0 |

Quadratic time → O(n²) Ex. for ( ) { for ( ) { } } 2 nested loops

Cubic time → O(n³) Ex:- 3 nested loops

Big - O Notation



$\Rightarrow O(1) < O(\log N) < O(N) < O(N \log N) < O(N^2) < O(N^3) < O(2^n) < O(N!)$

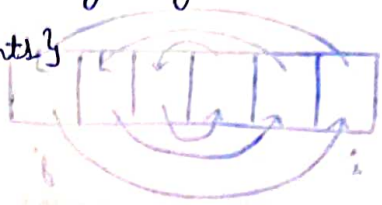
Lowest complexity — Highest complexity

Qu- Find Time Complexity :-

(a) $f(n) \rightarrow 2n^2 + 3n$

As Big-O denotes upper bound so we always ignore lower degree

$\Rightarrow O(n^2)$ { Also ignore constants }

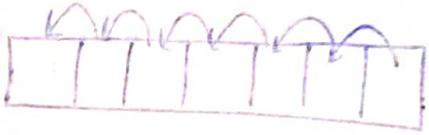


(b) $f(n) \rightarrow 4n^4 + 3n^3$

$\Rightarrow O(n^4)$

(c) $f(n) \rightarrow N^2 + \log N$

$\Rightarrow O(N^2)$



(d) $f(n) \rightarrow 3n^3 + 2n^2 + 5$

$\Rightarrow O(n^3)$

(e) $f(n) \rightarrow 12001$

$\Rightarrow O(1)$

(f) $f(n) \rightarrow \frac{n^3}{300}$

$\Rightarrow O(n^3)$

(g) $f(n) \rightarrow 5n^2 + \log n$

$\Rightarrow O(n^2)$

(h) $f(n) \rightarrow \frac{n}{4}$

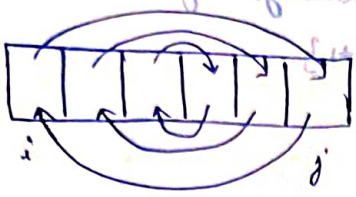
$\Rightarrow O(n)$

(i) $f(n) \rightarrow \frac{n+4}{4}$

$\Rightarrow O(n)$

Ex:- for (0 → n)
{
n iteration
⇒ O(n)
cout << "____";
}

Ex:- While reversing array



n length array
↳ $\frac{n}{2}$ swap
⇒ $O(\frac{n}{2})$ ⇒ O(n)

Because we don't consider constants

Ex:- Array linear search

n length array
Key

n comparisons
for (0 → n) {
.
}

⇒ O(n)

Qu- What is the time, space complexity of following code:

```
int a=0, b=0;
for (i=0; i<N; i++) {
    a = a + rand();
}
for (j=0; j<M; j++) {
    b = b + rand();
}
```

Assume that rand() is O(1) time, O(1) space function

⇒ O(N) + O(M)
⇒ O(N+M) (Time Complexity)

⇒ O(1) (Space)

Qu- What is the time, space complexity of following code:-

```
int a=0, b=0;
for (i=0; i<N; i++) {
```

```
for (j=0; j<N; j++) {
    a = a + j;
}
}
for (K=0; K<N; K++) {
    b = b + K;
}
```

$\Rightarrow O(N \times N) + O(N)$

$\Rightarrow O(N^2) + O(N)$ (Because upper bound we take)

$\Rightarrow \underline{O(N^2)}$ (Time) $\Rightarrow O(1)$ (Space)

Qu- What is the time complexity of the following code:-

```
int a=0;
for (i=0; i<N; i++) {
    for (j=N; j>i; j--) {
        a = a+i+j;
    }
}
```

for (0 → N) ⟶ N

{

for (N → i) ⟶ N

↓

N → 0

{

}

}

In case of Big-O hame worst se worst case sochna hai kyoki Big-O max. time Batata Hai kitna lag sakta hai

$\Rightarrow O(N \times N)$

$\Rightarrow \underline{O(N^2)}$

$\Rightarrow O(1)$ (Space)

$\underline{10^8 \text{ operation rule}}$ → Most of the modern machine can perform 10^8 operation / second

$\underline{\text{Constraints}}$: $1 \le n \le 10^6$

$1 \le n \le 1000$

⋮

anything

$\leq [10 \text{ -- } 11]$

$< [15 \text{ -- } 18]$

< 100

< 400

< 2000

$< 10^4$

$< 10^6$

$< 10^8$

Time Complexity

$O(n!) . O(n^6)$

$O(2^n * n^2)$

$O(n^4)$

$O(n^3)$

$O(n^2 * \log n)$

$O(n^2)$

$O(n \log n)$

$O(n)$, $O(\log n)$

Space Complexity

↳ Amount of memory taken by an algorithm

```
int a, b, c ;. ⟶ O(1)
```

Ex:-
```
func( )
{
    int arr[5] = {1, 2, 3, 4, 5}
}
```
⇒ S.C. → O(1)

Because array is fixed size

Ex:-
```
int n;
cin >> n;
vector<int> v(n);
```
⇒ S.C → O(n)

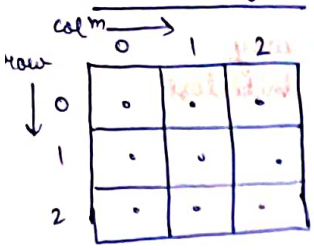
length → n

Ex:-
```
for (0 — n)
{
    vector<int> v(n);
    for (0 — n)
    {
    }
}
```
⇒ S.C. → O(n)

Kyoki Har loop me Baar baar utne hi size ka Ban raha Hai

Lecture (23)

2-D Arrays



3 row × 3 col

memory me store

Visualize aise karenge

Kuch mathematical formula use karke

$i \rightarrow$ row

$j \rightarrow$ col^m

formula $\Rightarrow$ $c \times i + j$ $\quad c \rightarrow$ Total col^m

Ex:- 1st row, 0th col^m $\Rightarrow$ $3 \times 1 + 0 = 3^{rd}$ index

2nd row, 1st col^m $\Rightarrow$ $3 \times 2 + 1 = 7^{th}$ index

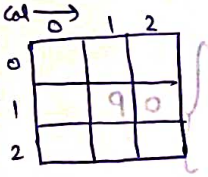
Creating 2-D array

```
int arr[10];
```

(linear array)

```
int arr[3][3];
```

row ↗ col ↖



(2-D array)

Taking input

(1-D array) `cin >> arr[i];`

(2-D array) `cin >> arr[i][j];`

Output:-

(1-D array) `cout << arr[i];`

(2-D array) `cout << arr[i][j];`

```
int arr[3][4];
for (int i=0; i<3; i++) {
    for(int j=0; j<4; j++) {
        cin >> arr[i][j];
    }
}
```

Takeind row-wise input

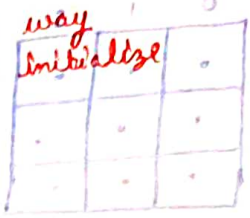
```
for (int i=0; i<4; i++) {
    for(int j=0; j<3; j++) {
        cin >> arr[j][i];
    }
}
```

Taking col^m-wise input

int arr[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 14, 16}; => Row wise dalenge

↳ Can be written like

int arr[3][4] = { {1,2,3,4}, {5,6,7,8}, {9,10,14,16} }; → Better way to initialize

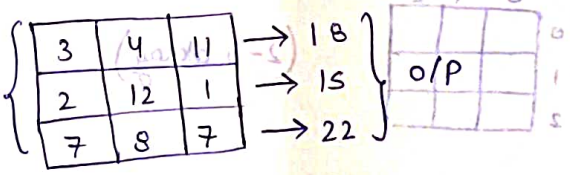


Qu- Linear Search in 2-D array

```
bool isPresent(int arr[][4], int target, int row, int col) {
    for (int row=0; row<3; row++) {
        for (int col=0; col<4; col++) {
            if (arr[row][col] == target) {
                return 1;
            }
        }
    }
    return 0;
}
```

↳ Find out why taking col^m is necessary in fxn

Qu- Row-wise sum,

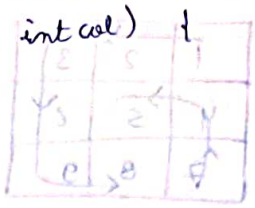



```
for (int m=0; m<i; m++) {
    int sum=0;
    for (int n=0; n<j; n++) {
        sum = sum + arr[m][n];
    }
    cout << sum << endl;
}
```

Qu- Largest Row which have maximum sum

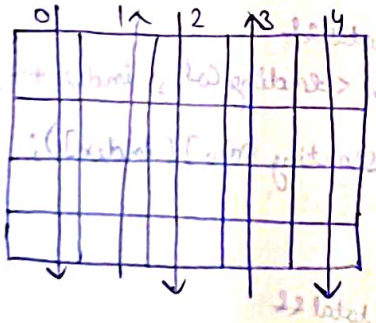
```
int maxSum = INT_MIN;
int rowIndex = -1;
for (int i=0; i<3; i++) {
    int sum=0;
    for (int j=0; j<4; j++) {
        sum = sum + arr[i][j];
    }
    if (sum > maxSum) {
        maxSum = sum;
        rowIndex = i;
    }
}
cout << "Row index " << rowIndex << " with max sum " << maxSum << endl;
```

```
int largestRowSum(int arr[][3], int row, int col) {
    int maxi = INT_MIN;
    int rowIndex = -1;
    for (int row=0; row<3; row++) {
        int sum=0;
        for (int col=0; col<3; col++) {
            sum += arr[row][col];
        }
        if (sum > maxi) {
            maxi = sum;
            rowIndex = row;
        }
    }
    cout << maxi << endl;
    return rowIndex;
}
```



Qu- Print like a wave, given 2-D array size (N X M), print the arr in a sine wave order i.e print the first column top to bottom, next column bottom to top and so on.

Soln



Observation:-

Col Index (odd) ⇒ Bottom to top

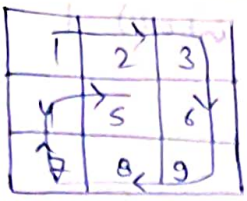
Col Index (even) ⇒ Top to Bottom

T.C. ⇒ O(nm)

```
for (int col=0; col<mCols; col++) {
    if (col&1) {
        for (int row = nRows-1; row>=0; row--) {
            cout << arr[row][col] << " ";
        }
    }
    else {
        for (int row = 0; row < nRows; row++) {
            cout << arr[row][col] << " ";
        }
    }
}
```

Leetcode 54

Qu- Spiral print



O/P => 1, 2, 3, 6, 9, 8, 7, 4, 5

Soln :- Approach :-

→ Print starting wali row

→ Ending col^n print karo

→ Ending row print karlo

→ Starting col^n print karlo

```
vector<int> spiralOrder(vector<vector<int>>& matrix) {
    vector<int> ans;
    int row = matrix.size();
    int col = matrix[0].size();

    int count = 0;

    int total = row * col;

    int startingRow = 0;
    int startingCol = 0;
    int endingRow = row - 1;
    int endingCol = col - 1;

    while (count < total) {
        for (int index = startingCol; index <= endingCol; index++) {
            ans.push_back(matrix[startingRow][index]);
            count++;
        }
        startingRow++;

        for (int index = startingRow; index <= endingRow; index++) {
            ans.push_back(matrix[index][endingCol]);
            count++;
        }
        endingCol--;

        for (int index = endingCol; index >= startingCol; index--) {
            ans.push_back(matrix[endingRow][index]);
            count++;
        }
        endingRow--;
```

count < total &&

```
for (int index = endingRow; index >= startingRow; index--) {
    ans.push_back(matrix[index][startingCol]);
    count++;
}
startingCol++;
}
return ans;
}
```

T.C ⇒ O(n+m)

Leetcode 48

Qu- Rotate matrix by 90°

Leetcode 74

Qu- Search a 2D Matrix (Sorted Matrix)

	0	1	2	3
0	1	3	5	7
1	10	11	16	20
2	23	30	34	60

⇒

0	1	2	3	4	5	6	7	8	9	10	11
1	3	5	7	10	11	16	20	23	30	34	60

```
Start = 0;
end = row * col - 1;
```

Case 1 :- if (arr[mid] == target) ↳ return mid

Case 2 :- if (arr[mid] < target) ↳ s = mid + 1;

Case 3 :- if (arr[mid] > target) ↳ e = mid - 1;

```
int element = matrix[mid/col][mid%col]
```

T.C ⇒ O(log n) ⇒ O(log (row * col))

Leetcode 240

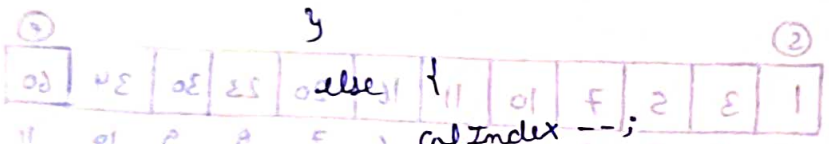
Qu- Search a 2D matrix II (row wise sorted) & (column wise sorted)

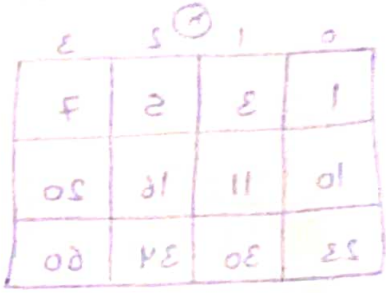
Case 1 :- arr[mid] == target ⇒ return mid

Case 2 :- arr[mid] < target ⇒ row++

Case 3 :- arr[mid] > target ⇒ col--;

```
int row = matrix.size();
int col = matrix[0].size();

int rowIndex = 0;
int colIndex = col - 1;

while (rowIndex < row && colIndex >= 0) {
    int element = matrix[rowIndex][colIndex];
    if (element == target) {
        return 1;
    }
    if (element < target) {
        rowIndex++;
    }
    else {
        colIndex--;
    }
}

return 0;
```





Lecture (24)

Maths for DSA

Leetcode 204

Qu- Count Primes, given an integer n, return the number of prime numbers that are strictly less than n.

```
class solution {
private:
    bool isPrime(int k) {
        if (k<=1) {
            return 0;
        }
        else {
            for (int i=2; i<k; i++) {
                if (k % i == 0) {
                    return 0;
                }
            }
        }
```

```
        return 1;
    }
}
public:
    int countPrimes(int n) {
        int count = 0;
        if (n<=1) {
            return 0;
        }
        else {
            for (int i=2; i<n; i++) {
                if (isPrime(i)) {
                    count++;
                }
            }
        }
        return count;
    }
};
```

⇒ Time Limit Exceeded in this solution

↳ T.C. ⇒ $O(n^2)$

Sieve of Eratosthenes

n = 40

✗ (2) (3) ✗ (5) ✗ (7) ✗ ✗ ✗
(11) ✗ (13) ✗ ✗ ✗ (17) ✗ (19) ✗
✗ ✗ (23) ✗ ✗ ✗ ✗ ✗ (29) ✗
(31) ✗ ✗ ✗ ✗ ✗ (37) ✗ ✗ ✗

① Mark every no' as a prime number
② Table me aa rahe hai unko non prime mark karde

Sol^n to previous question:-

```
class solution {
public:
    int countPrimes(int n) {
```

```
int cnt = 0;
vector<bool> prime(n+1, true);
prime[0] = prime[1] = false;
for (int i=2; i<n; i++) {
    if (prime[i]) {
        cnt++;
        for (int j=2*i; j<n; j=j+i) {
            prime[j] = 0;
        }
    }
}
return cnt;
}
};
```

T.C → n total ⇒ $\frac{n}{2} + \frac{n}{3} + \frac{n}{5} + \frac{n}{7} + \frac{n}{11} + \text{----}$

$= n\left(\underbrace{\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \text{----}}_{\text{H.P.}}\right)$

T.C. ⇒ $O(n * \log(\log n))$

{ H.W.
↳ Code segmented sieve }

Qu- Find GCD/HCF

Ex- 24 & 72

24 :- 2 × 2 × 2 × 3

72 :- 2 × 2 × 2 × 3 × 3

⇒ 2 × 2 × 2 × 3 = 24

Euclid's Algorithm

↳ $\boxed{gcd(a,b) = gcd(a-b, b) \Rightarrow gcd(a\%b, b)}$

Ex:- $gcd(72,24) = gcd(48,24) = gcd(24,24) = gcd(0,24)$

↳ ans = 24

```
int gcd (int a, int b) {
    if (a==0)
        return b;
    if (b==0)
        return a;
    while (a != b) {
        if (a>b) {
            a = a-b;
        }
        else {
            b = b-a;
        }
    }
    return a;
}
```

For lcm

$\boxed{lcm(a,b) * gcd(a,b) = a*b}$

Modulo Arithmetics

→ $a\%n \Rightarrow$ ans. = 0 to (n-1)

→ $(a+b)\%m = a\%m + b\%m$

→ $a\%m - b\%m = (a-b)\%m$

→ $a\%m * b\%m = (a*b)\%m$

Fast Exponentiation

$a^b \Rightarrow 2^{10}$ or 2^{11} (example)

a^b → $(a^{b/2})^2$ ⇒ if b is even

→ $(a^{b/2})^2 \times a$ ⇒ if b is odd

Qu- Modular Exponentiation, You are given a three integers 'X', 'N' and 'M'. Your task is to find ('X' ^ 'N') % 'M'. A^B is defined as A raised to power B and A % C is the remainder when A is divided by C.

```
int res = 1;
while (n > 0) {
    if (n & 1) {
        res = res * x;
    }
    x = x * x;
    n = n >> 1;
```

```
int modularExponentiation (int x, int n, int m) {
    int res = 1;
    while (n > 0) {
        if (n & 1) {
            res = (1LL * (res) * (x) % m) % m;
        }
        x = (1LL * (x) % m * (x) % m) % m;
        n = n >> 1;
    }
    return res;
}
```

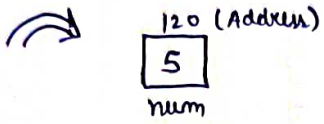
H.W.

① Read Articles given in description
② Pigeon hole Principle
③ Catalon number
④ Inclusion Exclusion principle
⑤ Factorial by using % m ⇒ (212!) % m , m = $10^9 + 7$

Lecture 25

Pointers in C++

int num = 5;



cout << num;

(Memory me Har value address se store Hogi to jab mai cout<< num bolunga to matlab wo address par point karega aur jo us address par value hai wo return karega)

Memory → Symbol table

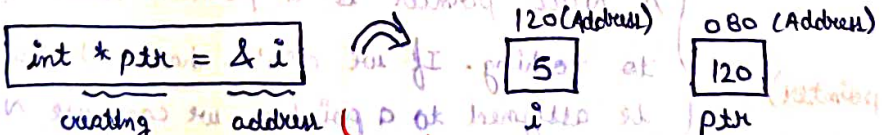
⇒ Symbol table is an important Data Structure created and maintained by the compiler in order to keep track of semantics of variables i.e. it stores information about the scope and binding information about names, information about instances of various entities such as variable and function names, classes, objects, etc.

⇒ Address is mapped by the symbol table

// Address of operator ⇒ & (cout << & num;)

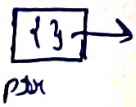
Address ⇒ 0x16b11F488 (Hexadecimal format)

Pointer ⇒ store addresses (a variable that stores the address of another variable)



(Declaring pointer at the same time)

int *ptr ⇒ pointer declare ← (Bad practice)



→ Garbage value

So, initialize at the same time

int *p (read as) p is a pointer to int data type

```
char ch = 'a';
char *p = &ch
int num = 5
int *p = &num;
```

} Type of pointer should be same (Datatype)

`cout << *p;` —— 5 (*p ⇒ Value at address pointed by p)

`cout << num;` —— 5

main()

a = 5

fun(a)

fun()

a = 5

5 (a) 5 (a) (Call by Value)

a++

5 (a) 6

---

ptr → 5 ← num

num++

`cout << num;` —— 6

`cout << *ptr;` —— 6

`cout << ptr;` —— print address of num

`cout << &ptr;` —— print address of pointer

⇒ Kisi bhi type ka data hai int, float, double, char, bhi Kaise me pointer ka size to same hi rahega kyoki wo address store karta hai (generally 8 bytes)

`int *p = 0;` (Null pointer) 0 / ptr

A NULL pointer is a pointer that is pointing to nothing. If we don't have the address to be assigned to a pointer, we can use NULL.

`cout << *p << endl;` ⇒ Segmentation fault

Better than → `int *p;` (points at some garbage address)

⇒ * is the dereference operator and can be read as "value-pointed to by"

```
int i = 5;
int *p = 0;
p = &i;
```

↷ second way to declare a pointer

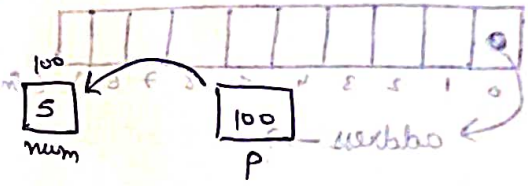
We can also assign null pointer as

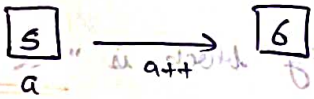
```
int *p = NULL;
```

```
cout << p << endl;   ——— 0x16d4c4488   (address of i)
cout << *p << endl;  ——— 5
int *q = &i;
cout << q << endl;   ——— 0x16d4c4488   (address of i)
cout << *q << endl;  ——— 5
```

```
int num = 5;
int a = num;
a++;
cout << num; ——— 5
```

```
int *p = &num;
int a = *p;
a++;
cout << num; ——— 5
```



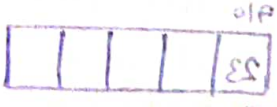


```
int *p = &num;
*p++;
cout << num; ——— 6
*p = (*p) * 5;
cout << num; ——— 30
```

Copying a pointer

```
int *q = p;
```

↷ Copying a pointer




```
cout << p << " - " << q << endl;   ——— 0x16d823488 - 0x16d823488
cout << *p << " - " << *q << endl; ——— 30 - 30
```

Pointer Arithmetic

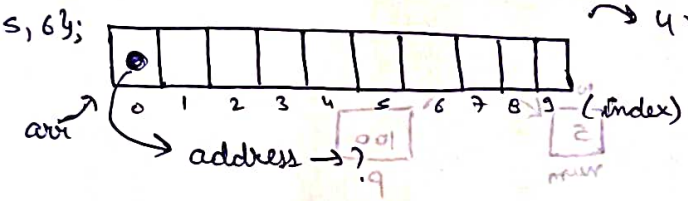
```
int i = 3;
int * t = &i;
cout << (*t)++ << endl;   ——— 3
cout << (*t) << endl;     ——— 4
cout << t << endl;        ——— 0x16fce746c
t = t + 1;                ——— 4 byte aage badh jayega (Because address hai)
cout << " after " << t << endl;  — 0x16fce7470
```

H.O.

① Read theory & practice problem on pointers from Code studio

Lecture (26)

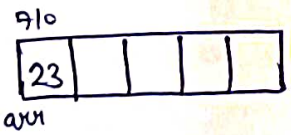
```
int arr[10] = {2, 5, 6};
```




```
cout << " Address of first memory block is " << arr << endl;   ——— 0x16f497410
cout << " Address of first memory block is " << &arr[0] << endl;  ——— 0x16f497410
cout << *arr << endl;   ——— (Value at 0th index) · 2
cout << (*arr + 1) << endl;   ——— 3   (Value at 0th location + 1)
cout << *(arr + 1) << endl;   ——— (Value at 1st location) 5
```

⇒ `arr[i] = *(arr + i)` or `i[arr] = *(i + arr)`

```
int arr[5];
int * p = &arr[0];
cout << *p << endl;   ——— 23
```



(20 Bytes)

(8 Bytes)

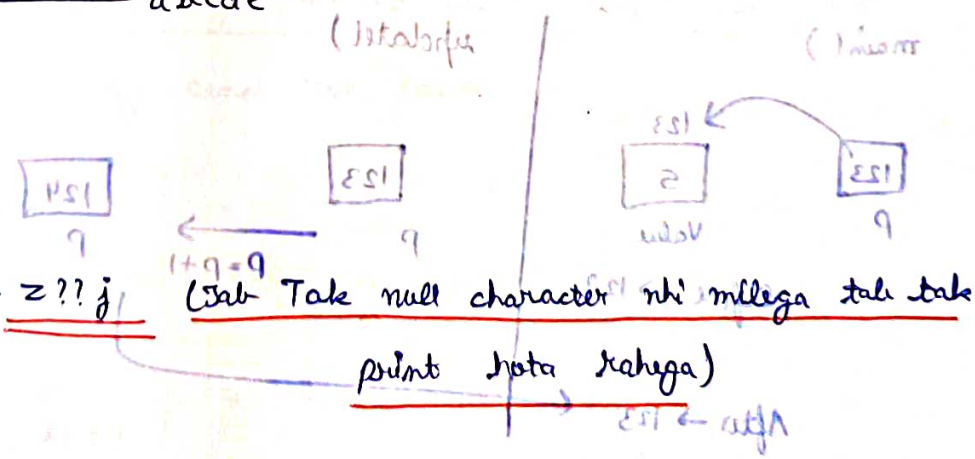
```
int temp [10] = {1,2};
cout << sizeof (temp) << endl;      ——— 40
cout << size of (* temp) << endl;   ——— 4
cout << size of (& temp) << endl;   ——— 8

int * ptr = & temp [0];
cout << sizeof (ptr) << endl ;      ——— 8
cout << size of (*ptr) << endl;     ——— 4
cout << sizeof (& ptr) << endl;     ——— 8
```

⇒ The content of Symbol table cannot be changed.

```
int arr [10];
arr = arr + 1;   ⇒ array type is not assignable
```

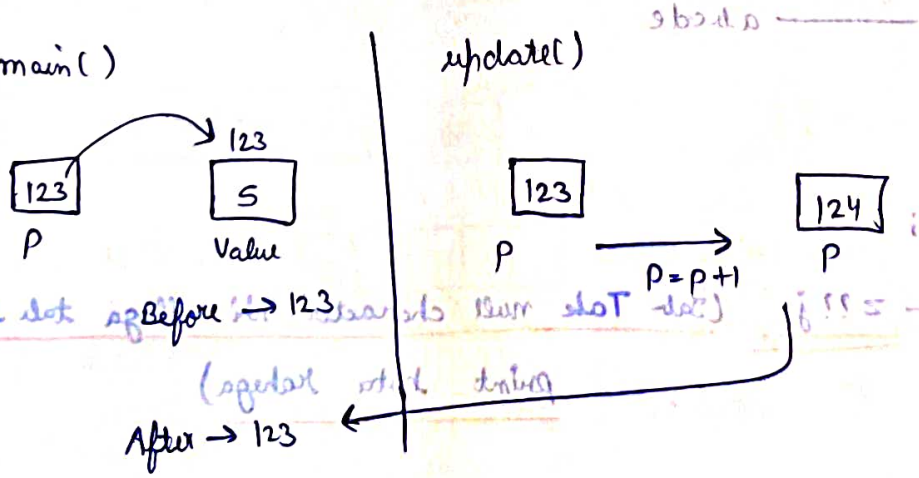
Char arrays

```
int arr [5] = {1,2,3,4,5};
int ch [6] = "abcde";
cout << arr << endl;    ——— 0x16b3db410
cout << ch << endl;     ——— abcde   (Working of char array is different)

char * c = & ch [0];
cout << c << endl;      ——— abcde

char temp = 'z';
char * p = & temp;
cout << p ;             ——— z?? j   (Tab Take null character nhi milega tab tak print hota rahega)
```



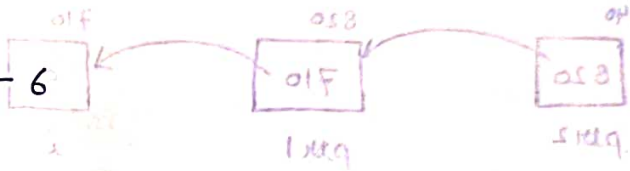
Functions

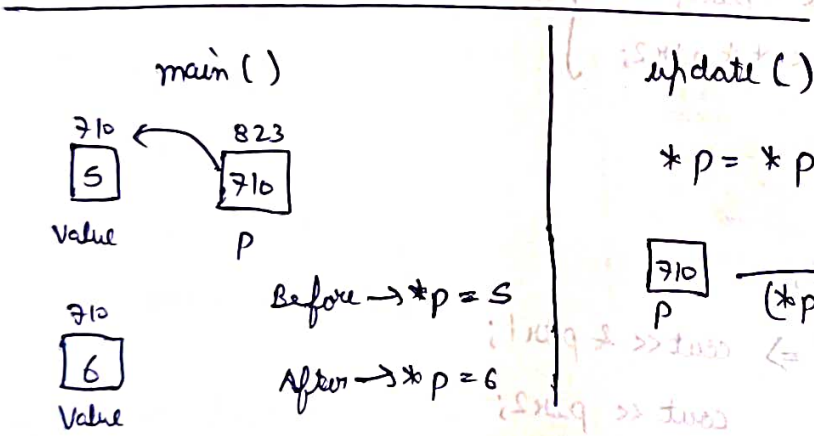
```
void print (int *p) {
    cout << *p << endl;
}

void update (int *p) {
    p = p + 1;
    cout << "inside " << p << endl;
}

int main () {
    int value = 5;
    int *p = &value;
    print (p);
    cout << "Before " << p << endl;
    update (p);
    cout << "After " << p << endl;
    return 0;
}
```



But we can change value

```
void update (int *p) {
    *p = *p + 1;
}

int main () {
    int value = 5;
    int *p = &value;
    cout << *p << endl;  ——— 5
    update (p);
    cout << *p << endl;  ——— 6
    return 0;
}
```





⇒ function me jab array pass karenge To wo as a pointer jaata hai.

Benefit → Aap ek part of array send kar sakte ho

```
int getSum (int arr[], int n) {
    cout << "Size:" << sizeof (arr) << endl;  ——— 8
    int sum=0;
    for (int i=0; i<n; i++) {
        sum += arr[i];
    }
    return sum;
}
```

```
int arr[6] = {1,2,3,4,5,8};
cout << getSum(arr+3, 3) << endl;  ⇒ (4+5+8) = 17
}
```

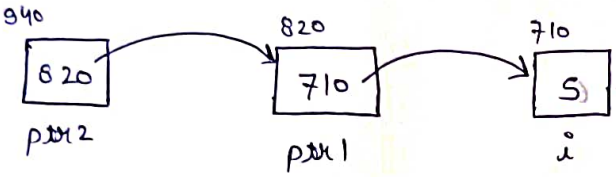
H.W.

Practice from Code Studio

Lecture (27)

Double Pointer

```
int i = 5;
int *ptr1 = &i;
int **ptr2 = &ptr1;   ⇒ Double pointer
```



⇒ For accessing value of i ⇒ cout << i;
cout << *ptr1;
cout << **ptr2;
} All will give output 5

⇒ For printing 710 ⇒ cout << &i;
(address of i) cout << ptr1;
cout << *ptr2;

⇒ For printing address of ptr1 (820) ⇒ cout << &ptr1;
cout << ptr2;

Double Pointers and Functions

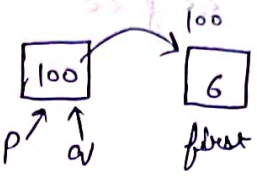
```
void update(int **p) {
    p = p+1;      ⇒ No change
    *p = *p+1;    ⇒ change in value of ptr1
    **p = **p+1;  ⇒ change in value of i
}
```

Qu- int first = 8;
int second = 18;
int *ptr = &second;
*ptr = 9;
cout << first << " " << second << endl;

Ans.
Output:-
8 9

Qu- int first = 6;
int *p = &first;
int *q = p; (Copying a pointer)
(*q)++;
cout << first << endl;

Ans.



Output:- 7

Qu- int first = 8;
int *p = &first;
cout << (*p)++ << " ";
cout << first << endl;

Ans.
Output:-
8 9

Qu- int *p = 0;
int first = 110;
*p = first;
cout << *p << endl;

Ans.
Segmentation fault
in case of null pointer
int *p = 0;
p = &i;

Qu- int first = 8;
int second = 11;
int *third = &second;
first = *third;
*third = *third + 2;
cout << first << " " << second << endl;

Ans.
11 13

Qu-
```
float f = 12.5;
float p = 21.5;
float *ptr = &f;
(*ptr)++;
*ptr = p;
cout << *ptr << " " << f << " " << p << endl;
```

Output:-

21.5 21.5 21.5

Qu-
```
int arr[5];
int *ptr;
cout << sizeof(arr) << " " << sizeof(ptr) << endl;
```

Output:-

20 8

Qu-
```
int arr[] = {11, 21, 13, 14};
cout << *(arr) << " " << *(arr+1) << endl;
```

Output:-

11 21

Qu-
```
int arr[6] = {11, 12, 31};
cout << arr << " " << &arr << endl;
```

Output:-

0x16d283450 0x16d283450

(Prints address of 1st location)

Qu-
```
int arr[6] = {11, 21, 13};
cout << (arr+1) << endl;
```

Output:-

Prints address of 2nd location

Qu-
```
int arr[6] = {11, 21, 31};
int *p = arr;
cout << p[2] << endl;
```

Output:-

p[2] => *(p+2)

Ans- 31

Qu-
```
int arr[] = {11, 12, 13, 14, 15};
cout << *(arr) << " " << *(arr+3);
```

Output:-

11 14

Qu-
```
int arr[] = {11, 21, 31, 41};
int *ptr = arr++;
cout << *ptr << endl;
```

Output:-

Error (Because array can't be updated)

Qu-
```
char ch = 'a';
char *ptr = &ch;
ch++;
cout << *ptr << endl;
```

Output:-

b

Qu-
```
char arr[] = "abcde";
char *p = &arr[0];
cout << p << endl;
```

Output:-

abcde (functioning of char array is different)

Qu-
```
char arr[] = "abcde";
char *p = &arr[0];
p++;
cout << p << endl;
```

Output:-

bcde

Qu-
```
char str[] = "babbar";
char *p = str;
cout << str[0] << " " << p[0] << endl;
```

Output:-

b b

Qu-
```
void update (int *p) {
    *p = (*p) * 2;
}
int main () {
    int i = 10;
    update (&i);
    cout << i << endl;
}
```

Output:-

20

Qu-
```
int first = 110;
int *p = &first;
int **q = &p;
int second = (**q)++ + 9;
cout << first << " " << second << endl;
```

Output:-

111 119

Qu-

```
int first = 100;
int * p = & first;
int ** q = & p;
int second = ++(**q);
int * x = *q;
++(*x);
cout << first << " " << second << endl;
```

Output:-

102 101

Qu-

```
void increment(int **p) {
    ++(**p);
}
int main() {
    int num = 110;
    int *ptr = & num;
    increment(& ptr);
    cout << num << endl;
}
```

Output:-

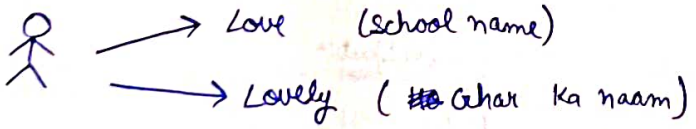
111

Hw.

Complete pointers from Code Studio

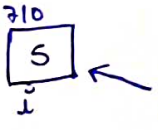
Lecture (28)

Reference Variable



→ Same memory but different names

```
int i = 5;
int & j = i;   (creating a reference variable)
```



```
int main() {
    int i = 5;
    int &j = i;
    cout << i;      ——— 5
    i++;
    cout << i;      ——— 6
    j++;
    cout << i;      ——— 7
    cout << j;      ——— 7
    return 0;
}
```

Why Reference Variable ?

→ In Pass by Value, new memory is created i.e. copy is created

→ In Pass by Reference, the function variable points at the same memory

Pass by Reference

```
int update (int &n) {
    ========
    ========
    ========
}
```

Return by Reference

```
int & update (int n) {
    int a = 10
    int &ans = a;
    return ans;
}
```

a → local variable

so Bad practice

Warning will be there

same problem with return by pointer

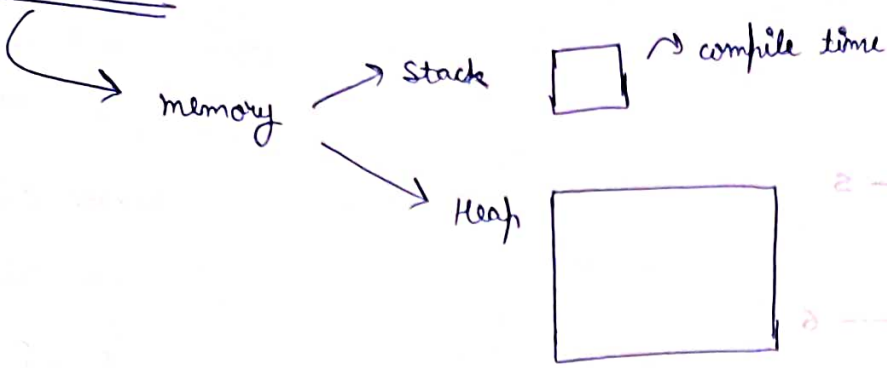
Array

```
int n;
cin >> n;
int arr[n];   ⇒ Bad Practice
```

Because isse array ka size runtime par pata chalega jabki compile time par pata chal jaana chaahiye

```
int arr[10000];   ⇒ Better than previous
```

Program start



⇒ Because runtime me stack utni hi memory leke aata hai aur agar maine variable sized array banakar uska size aisa daal diya jo stack ki memory se bhi bada ho to program crash kar jayega.

So, How to make Variable sized array?

↳ By using Heap Memory

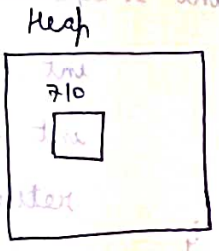
↳ By using 'new' keyword

When using stack memory ⇒ static memory allocation

When using Heap memory ⇒ Dynamic memory allocation

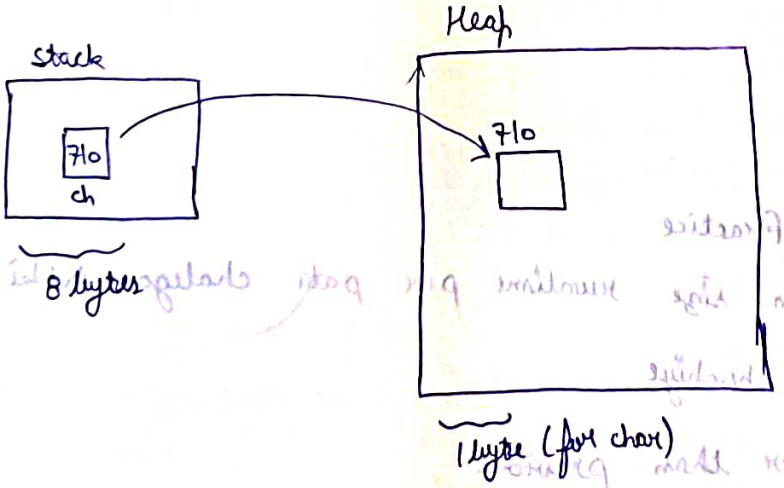
new char;

↳ returns address



char *ch = new char;

↳ Using pointer for accessing the value



Total 9 bytes

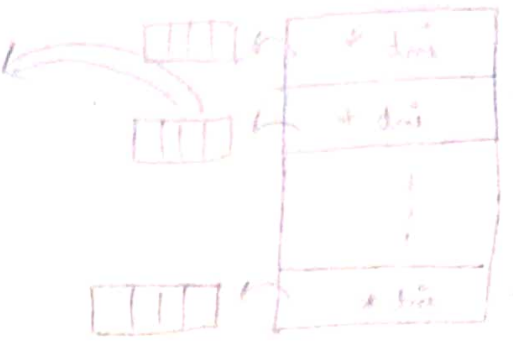
For creating array

```
int * arr = new int [5];
```

8 bytes + 20 bytes

Total = 28 bytes

```
int n;
cin >> n;
int * arr = new int [n];
```



Difference

Static

① `int arr [50]`

50 × 4 = 200 byte (stack)

② Memory automatically release/deleted

Dynamic

① `int * arr = new int [50]`

8 byte pointer (stack) + 50 × 4 = 200 byte (heap)

Total = 208 byte

② Manually release by using 'delete' keyword

```
int * i = new int;
delete i;
```

For array

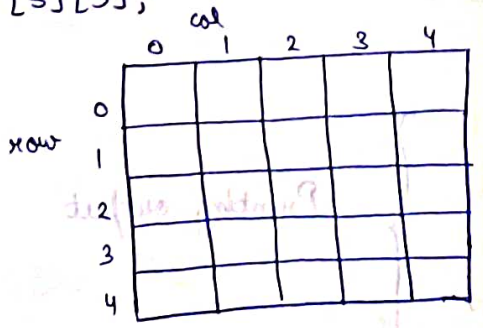
```
int * arr = new int [50];
delete [] arr;
```

H.W.

Read Void Pointer, Address Typecasting

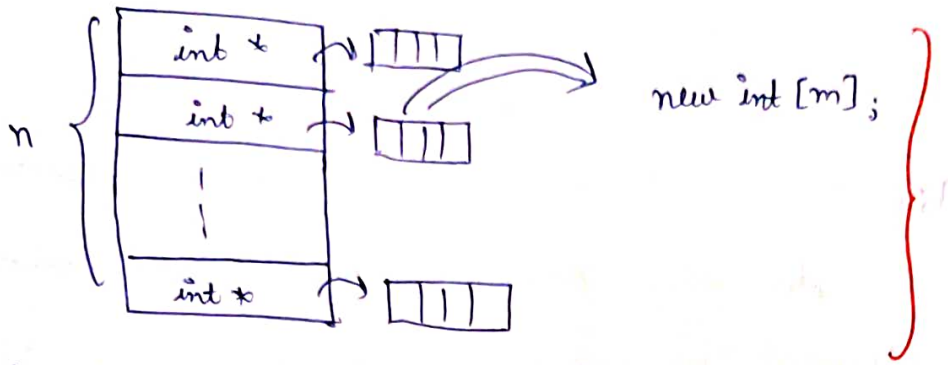
Lecture (29)

`int arr [5][5];` ← By statically allocated



Dynamic Memory Allocation of 2D Array

```
int ** arr = new int *[n];
```




```
for ( ) {
    arr [i] = new int [n];
}
```

Implementation

```
int main() {
    int m, n;
    cin >>m>>n;
    int ** arr = new int *[m];
    for ( int i=0; i<m; i++) {
        arr [i]= new int [n];
    }
```

Creation of 2-D array

```
    for (int i=0; i<m; i++) {
        for ( int j=0; j<n; j++) {
            cin >> arr [i] [j];
        }
    }
```

Taking input

```
    for (int i=0; i<m; i++) {
        for (int j=0; j<n; j++) {
            cout<< arr [i] [j]<< " ";
        } cout<< endl;
    }
}
```

Printing output

For releasing / deleting memory

```
for (int i=0; i<m; i++) {
    delete[] arr[i];
}
delete[] arr;
```

H.W.

Create Jaggered Array dynamically

Lecture (30)

Macros

define ⟶ Macro create

include <iostream>

↳ preprocessor directive

Let's assume in my code I have used pi = 3.14 many times now if I have to change the value to 3.29 then what should I do, Kya mai Har jagah jaake change karunga value ya kuch solution Hai

1st sol^n

Maine starting me hi ek variable create kar diya

double pi = 3.14;

Aur mai har jagah 'pi' use karunga aur agar value change karni hui to starting me hi kar dunga but iske nuksaan hai jaise

↳ Storage use in creating a variable so performance hinder

2nd sol^n

By creating a (macro) ↳ a piece of code in a program that is replaced by value of macro

define PI 3.14

(Better than 1st sol^n) ↳ Because extra memory ki need nhi lene padegi

→ Macro can't be updated

H.W.

Read GFG macro article in description

Global Variable

Variable ⟶ share (By reference variable)

↳ Also we can share b/w functions by global variable

So, you can create a variable outside all functions as well as main function just like creating normal variable.

But creating global variable is bad practice. Because agar kisi bhi function ne change kar diya To vo change har jagah ho jaayega.

So, for sharing use reference variable

Inline Functions

↳ used to reduce the function calls overhead

```
int getMax(int &a, int &b) {
    return (a>b) ? a : b;
}

int main() {
    int a = 1, b = 2;
    int ans = 0;
    ans = getMax(a, b);
    cout << ans << endl;
    a = a+3;
    b = b+1;
    ans = getMax(a,b);
    cout << ans << endl;
}
```

For making this as inline function
if function body is single line
if function body (2-3 lines) ⇒ may or may not
if function body > 3 lines ⇒ can't convert in inline function

```
inline getMax(int &a, int &b) {
    return (a>b) ? a : b;
}
```

Default Arguments

```
void print(int arr[], int n, int start=0) {
    for (int i=start; i<n; i++) {
        cout<< arr[i] << endl;
    }
}

int main() {
    int arr[5] = {1, 4, 7, 8, 9}
    int size = 5;
    print(arr, size);
    cout<< endl;
    print(arr, size, 2);

    return 0;
}
```

(Here start is the default argument i.e. start is optional)

Output:-

```
1
4
7
8
9

7
8
9
```

Cond" of default argument ⇒ Rightmost argument se shuru hoga

```
void trial(int a=0, int b)   ✗
void trial(int a, int b=0)   ✓
```

H.W.

Read constant Variable and documentation from Code studio